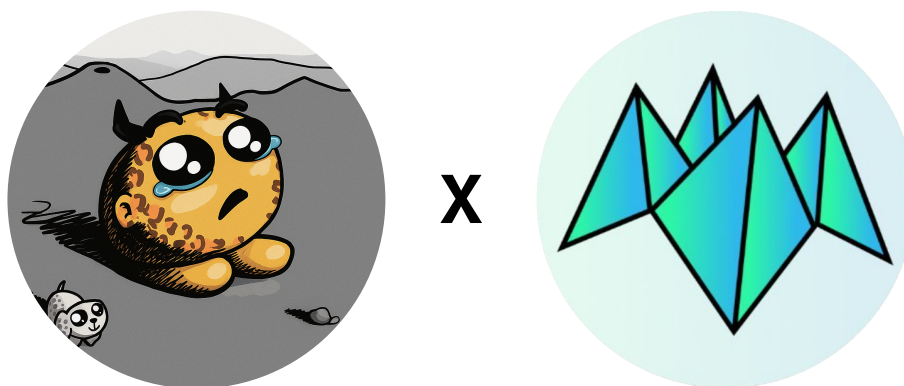

Origami Finance

Origami Portfolio of Assets Liabilities

Security Review

By Jacopod Audits



Auditors

Lead Security Researcher: Jacobo Lansac (@Jacopod)

February 8, 2026

Contents

1	Introduction	2
2	About Jacopod	2
3	About Origami Finance	2
4	Disclaimer	2
5	Risk classification	2
5.1	Impact	3
5.2	Likelihood	3
5.3	Action required for severity levels	3
6	Executive Summary	3
7	Architecture review	4
7.1	OPAL in a nutshell	4
7.1.1	Mental model	4
7.1.2	Main contracts	4
7.1.3	Entry Points	4
7.1.4	Architecture assessment	4
8	Findings	6
8.1	Informationals	6

1 Introduction

A time-boxed review of **Origami Portfolio of Assets Liabilities** smart contracts, developed by [Origami Finance](#). The focus of this review is to identify security vulnerabilities, explain their root-cause and provide solutions to mitigate the risk.

Gas optimizations are not the main focus but will also be identified if found.

2 About Jacopod

Jacopod Audits provides thorough security assessments for smart contract applications, combining deep manual review with advanced testing techniques like fuzzing and invariant testing to identify and mitigate potential vulnerabilities before deployment across diverse DeFi protocols.

Highlights:

- Ranked #6 in all-time Hats Finance ([Leaderboard](#)).
- More than \$200M TVL secured: [Origami Finance](#), (\$138M TVL), [TempleDAO](#), (\$60M TVL), [PinLinkAi](#), (\$21M TVL), [Vectorreserve](#), (\$45M peak TVL, now inactive).
- Recurring engagements with [Origami Finance](#) and [TempleDAO](#).

Contact for audits:

- X: [@jacopod](#)
- Telegram: [@jacopod_eth](#)
- Check the [Complete Audit Portfolio](#).

3 About Origami Finance

[Origami Finance](#) is a novel tokenised leverage protocol on Ethereum and Berachain. Fully integrated with third-party lenders, Origami allows users to achieve non-custodial portfolio exposure to popular looping and other yield strategies via a familiar vault UX. Origami vaults are fully automated to eliminate tedious reward harvesting, maximise capital efficiency, and minimise liquidation risk for the underlying position.

4 Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time and resource-bound effort to find as many vulnerabilities as possible, but there is no guarantee that all issues will be found. This security review does not guarantee against a hack. Any modifications to the code will require a new security review.

This review does not focus on the correctness of the happy paths. Instead, it aims to identify potential security vulnerabilities and attack vectors derived from an unexpected and harmful usage of the contracts. The devs are ultimately responsible for the correctness of the code and its intended functionality.

A security review is not an endorsement of the underlying business or product and can never be taken as a guarantee that the protocol is bug-free.

5 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

5.1 Impact

- **High:** leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium:** only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low:** can lead to unexpected behavior with some of the protocol's functionalities that are not so critical.

5.2 Likelihood

- **High:** almost certain to happen, easy to perform or highly incentivized.
- **Medium:** only conditionally possible or incentivized, but still relatively likely
- **Low:** requires multiple unlikely conditions, or little-to-no incentive

5.3 Action required for severity levels

- **Critical:** Must fix as soon as possible (if already deployed)
- **High:** Must fix (before deployment if not already deployed)
- **Medium:** Should fix
- **Low:** Could fix

6 Executive Summary

Scope details

Project Name	Origami Finance
Type of Project	Leveraged Yield Farming
Repository	TempleDAO/origami/
Review commit	cd1c7051450c5f7eb73f501c8755d56ee748937a
Mitigation commit	cd1c7051450c5f7eb73f501c8755d56ee748937a
Audit Timeline	2025-12-02 - 2026-01-29
Methods	Manual Review, Foundry, Testing

Files in scope	nSLOC
contracts/investments/opal/OpalAdapterFactory.sol	66
contracts/investments/opal/OpalManager.sol	433
contracts/investments/opal/OpalMaxJoinExitLib.sol	163
contracts/investments/opal/OpalVault.sol	194
contracts/investments/opal/adapters/OpalAdapterAaveV3.sol	467
contracts/investments/opal/adapters/OpalAdapterBase.sol	94
Total	1417

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	0

7 Architecture review

7.1 OPAL in a nutshell

OPAL (Origami Portfolio of Assets & Liabilities) is a tokenized balance-sheet vault that issues a single ERC-20 share representing a managed portfolio of on-chain positions across multiple venues. Unlike typical vaults that only track assets, OPAL models both assets and liabilities, enabling safe leverage, debt roll-overs, and cross-venue rebalances under one token.

- **Vault:** ERC-20 shares representing ownership of the vault. Time-based AUM fees (capped at 1,000 bps/year).
- **Manager:** Aggregates tokens, computes `maxJoin/maxExit`, performs join/exit allocation across adapters, and runs permissioned atomic multicall bundles.
- **Adapters:** Minimal Solady clones with immutable args, restricted to being called by the Manager. They are the interface connecting the manager to multiple DeFi protocols (Aave, Morpho, Euler).

7.1.1 Mental model

- **Users** join/exit via the Vault → Vault delegates to the Manager → Manager splits work across Adapters proportionally to their existing balances and limits.
- **Governance/operators** perform rebalances via `Manager.multicall([...])` using approved plugins; users remain abstracted from underlying venue details.

7.1.2 Main contracts

- `OpalVault` — ERC20 share token + balance sheet vault. User-facing entry/exit.
- `OpalManager` — Orchestrator/bundler. Allocates assets/liabilities proportionally across adapters. No direct storage writes on join/exit.
- Adapters (`OpalAdapterAaveV3`, `OpalAdapterEuler`, `OpalAdapterMorpho`) — Protocol-specific adapters to supply collateral and borrow liabilities from DeFi protocols.
- `OpalAdapterFactory` — Creates adapter clones with immutable args stored in bytecode.

7.1.3 Entry Points

The `OpalManager` is mostly permissioned and the only user entrypoints are in the `OpalVault`. These entrypoints are inspired on the typical ERC4626, but with the necessary adjustments to support multiple assets and liabilities:

- `joinWithToken()` : Deposit specific token amount → receive shares
- `joinWithShares()` : specific share amount → pay proportional assets, receive proportional liabilities
- `exitWithToken()` : Burn shares for specific token amount (asset or liability)
- `exitWithShares()` : specific shares → receive proportional assets and repay liabilities

7.1.4 Architecture assessment

- The architecture is very solid, with isolated components for security.
- The codebase has been clearly written with a security-first approach.
- The architecture also aims at being as gas efficient as possible for a good UX, without compromising security

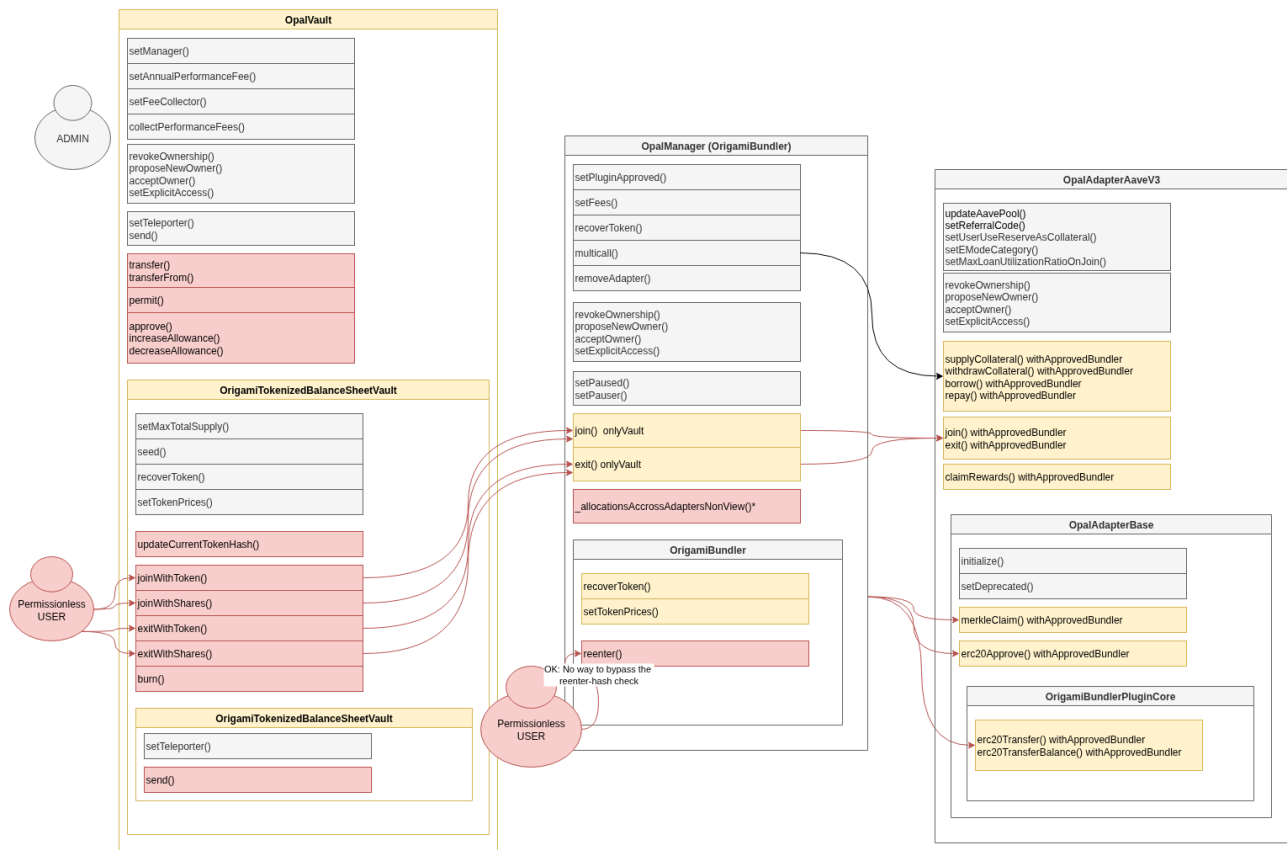


Figure 1: OPAL architecture overview

8 Findings

8.1 Informationals

Description

The function `_allocateToAdapter()` allows proportional assets (and liabilities) to round down to zero during small deposits, particularly when the smallest-allocation adapter uses tokens with fewer decimals (e.g., USDC with 6 decimals, WBTC with 8 decimals). This happens at the `mulDiv` operation, which can result in zero allocation to specific adapters despite having a non-zero balance, while other adapters receive non-zero amounts.

Repeated small deposits can gradually skew the allocation ratio between adapters, deviating from the intended proportional distribution. While the function correctly allows zero allocation for adapters with no previous allocation, it also permits zero allocation for adapters that should receive proportional shares based on their existing allocation. The current implementation lacks validation to ensure that adapters with existing allocations receive at least a minimum non-zero amount.

Another variation of the same issue is that an attacker can donate some small amount of assets to an adapter that has 0 allocation, but is not deprecated. This can create a very annoying UX for users who have to get hold of some dust amount for those assets that received donations.

Likelihood: Low

Impact: Info

An attacker could imbalance adapters outside the desired ratio by making multiple small deposits that round down one adapter allocation. However, the current protections wouldn't allow the adapter to go outside the safe LTV ratio, so there is no immediate risk.

Mitigation

Require a minimum allocation threshold check in `_allocateToAdapter()` for adapters with non-zero allocation:

```
// In _allocateToAdapter() after calculating proportional amount
uint256 proportionalAmount = totalAmount.mulDiv(adapterAllocation, totalAllocation);

// If adapter has existing allocation, enforce minimum non-zero amount
if (adapters[i].currentAllocation > 0 && proportionalAmount == 0) {
    revert InsufficientDepositAmount();
}
```

However, I am not sure I would implement this fix, because that could lead to bad UX in some cases with undesired reverts. And after all, small deviations from the ratio between adapters are expected to be adjusted by overlord rebalances.

Team response: Acknowledged